DSA - 1D Array

Song : Suspicious Minds
- Elvis Presley

Topics To Cover :
- Basics of 1D array
- Reverse array
- Rotate Array
- Space Complexity

Hi Everyone!!!

Arrays (in C++ and Java)
- Static length
- Homogenous data
- eg: int arr[10]
  - int ↑ datatype
  - 10 ↑ size

Python Lists
- Dynamic length
- Heterogenous data
- eg: li = [ ]
- Index starts from 0

Quiz : Index for the first and last ele

li =	5	12	20	19	13	24	22	30	40
	0	1	2	3	4	5	6	7	8
	↑ first								↑ last (N-1)

N = 9

```
def print_ar(li):
    for i in range(len(li)):
        print(li[i])
```

No. of iter required = N → no. of ele

TC: $\Theta(N)$

↳ Time to access ele : constant time = $\Theta(1)$

* Time to access an ith ele in a list → $\Theta(1)$
* To access all ele in an ar of size N → $N * \underbrace{\text{access time}}_{\text{constant}}$
$= \Theta(N)$

Q. Reverse the given list:

ar :

1	2	3	4	5
0	1	2	3	4

N = 5

ans :

5	4	3	2	1
0	1	2	3	4

Brute force: ① Using slicing, create a new list and return it.

```
def rev_list(li):
    rev_li = li[::-1]
    return rev_li
```

→ Creates a new list during slicing.

TC: $\Theta(N)$
↑ slicing operation

Space → A new ar is created.
SC: $\Theta(N)$

Reverse using 2 pointer approach.

i, j

1	2	3	4	5
0	1	2	3	4

i, j

5	2	3	4	1
0	1	2	3	4

i j

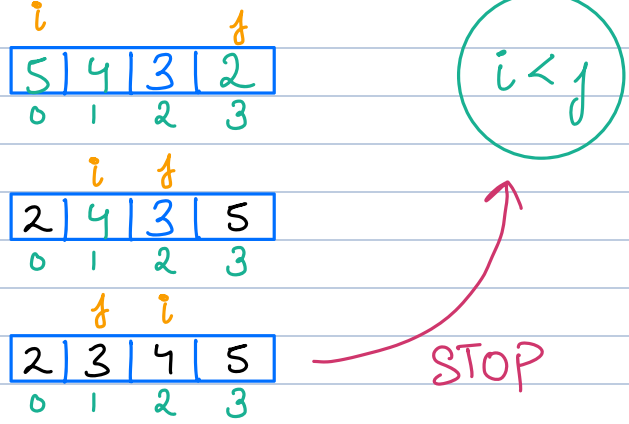
5	4	3	2	1
0	1	2	3	4

*① Swap the first and the last ele and move the pointers closer

② Stop if pointers meet or cross eachother.

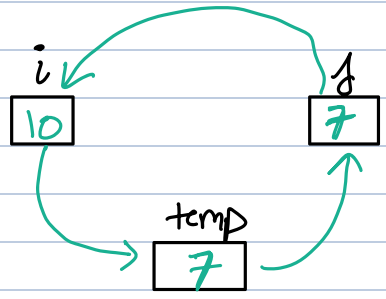
move i & j till $i < j$

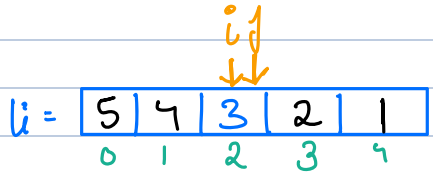
overlap ∴ stop.




```
def rev_list (li):
    i = 0
    j = len(li) - 1
    while (i < j):
        temp = li[i]
        li[i] = li[j]
        li[j] = temp
        i += 1
        j -= 1
    return li
```

swap





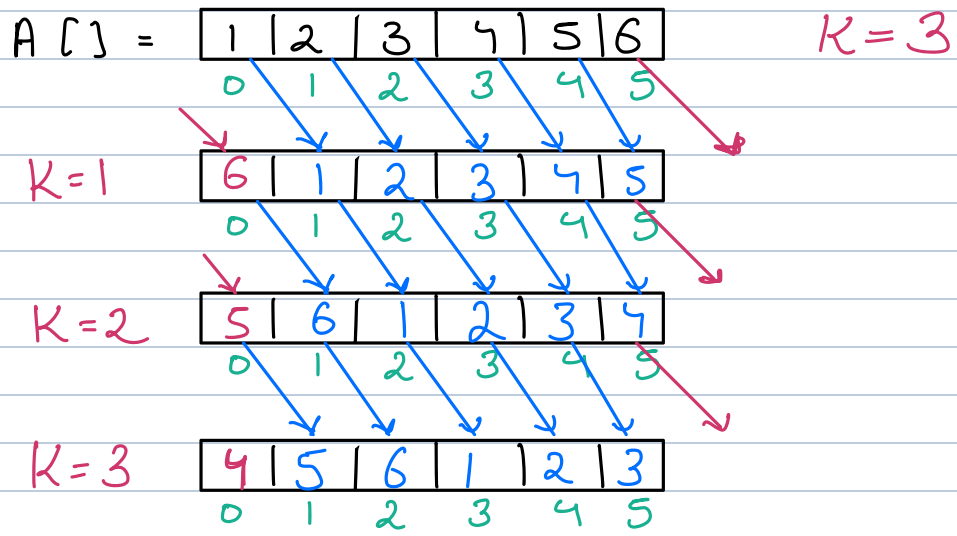
temp = 2

Iteration = $\frac{N}{2}$ TC: $\Theta(N)$

Space = No new array created.

SC : $\Theta(1)$

Q. Given an ar of len N, we wish to rotate the array in clockwise manner.



eg: li = [2, 5, 6, 8] K = 2 Ans = [6, 8, 2, 5]

K=1 [8, 2, 5, 6]

K=2 [6, 8, 2, 5]

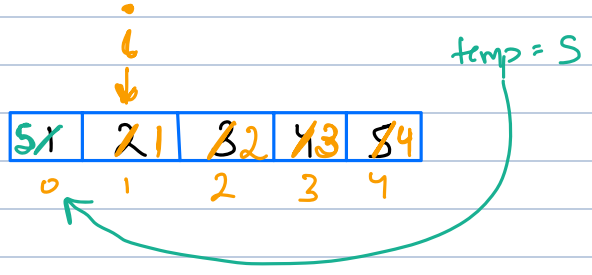
Brute force:
1. Save the last ele in a var
2. Move all ele forward
3. Put the saved var at index 0

li = [2, 5, 6, 8] temp = 8

li = [2, 5, 6, 6]

li = [2, 5, 5, 6]

li = [2, 2, 5, 6]




```
def rotate(li, K):
    for _ in range(K):
        temp = li[-1]
        for i in range(len(li)-1, 0, -1):
            li[i] = li[i-1]
        li[0] = temp
    return li
```

start end jump

Iteration: K * N-1 itrs

TC: $\Theta(KN)$

Path to optimisation

Q. Reverse ar[n] from s to e

ar : | 5 | 8 | 6 | 5 | 1 | 4 | 2 |
0 1 2 3 4 5 6

s = 2 e = 5

[5, 8, 4, 1, 5, 6, 2]

```
def rev_list (li, s, e):
    i = s
    j = e
    while (i < j):
        temp = li[i]
        li[i] = li[j]
        li[j] = temp
        i += 1
        j -= 1
    return li
```

swap

Optimisation :

A [ ] = | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
0 1 2 3 4 5 6

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | → rev_list (li, 0, N-1)
K N-1

| 7 | 6 | 5 | 1 | 2 | 3 | 4 | → rev_list (li, K, N-1)

| 5 | 6 | 7 | 1 | 2 | 3 | 4 | → rev_list (li, 0, K-1)

K = 3 | 5 | 6 | 7 | 1 | 2 | 3 | 4 |
0 1 2 3 4 5 6
K-1 K

```
def rotate (li, K):
    N = len (li)
    rev_list (li, 0, N-1)
    rev_list (li, K, N-1)
    rev_list (li, 0, K-1)
    return rev_list
```

Iteration = N + N + N = 3N

TC: $\Theta(N)$

SC: $\Theta(1)$

Space Complexity

```
fn (int x) {
    int m;   → 4 bytes
    int y;   → 4 bytes
    long z;  → 8 bytes
}
```

space = 16 bytes

↓

SC: $\Theta(1)$ → constant

Quiz:

```
def fn(N):
    arr = [0]*10          → 10 ints saved → Constant
    x, y, z = 1, 2, 3     → Constant
    ar_dynamic = [0]*N    → arr of size N.
```

Space : arr (constant) + x, y, z (Constant) + ar_dynamic

$= 1 + 1 + N$

$= \Theta(N)$

Quiz:

```
def func(N):
    x = N
    y = x*x
    z = x + y
    ar = [0]*N            → li of size N
    matrix = []
    for _ in range(N):
        row = []
        for _ in range(N):
            row.append(0)
        matrix.append(row)
```

Constants

row of size N

↳ We create N rows of size N and append to matrix.

Space $= N + N^2 = \Theta(N^2)$

TC: $\Theta(N^2)$